Projekt 3: overblik over segmenteringspipelinen

Gustav Bellaiche (kode skrevet med Claude Code)

4. november 2026

Kort fortalt

Hele koden til del 1 (fuld supervision på PH2 og DRIVE) og del 2 (punkt-klik på PH2) er skrevet og testet end-to-end på CPU. Den ligger på branchen `segmentation-pipeline` i `Colmeray/idlcv_project_3` og er **endnu ikke merget ind i `main`**. Der er **ikke kørt rigtig træning på GPU endnu**, så der findes ingen resultater til rapporten.

1 Datasættene

Begge ligger i `/dtu/datasets1/02516`. Tallene er målt direkte på filerne.

	PH2	**DRIVE**
Domæne	Dermoskopi af hudlæsioner	Fundusfotos af nethinden
Target	`IMDxxx_lesion.bmp`	`training/1st_manual/*.gif` (blodkar)
Billeder med target	200	20
Opløsning	ca. 767×576 (varierer lidt)	565×584
Andel forgrund	32 % (std. 23 %)	8,6 % (12,5 % inden for FOV)
Andre masker	ROI-masker for 27 billeder (ignoreres)	FOV-maske (`mask/`)

De to slags masker i DRIVE:

- `1st_manual` er blodkar-annotationen og dermed det, modellen skal forudsige.
- `mask` er en FOV-maske: den hvide cirkel, hvor der er nethinde. Den bruges kun til at regne metrikker inden for synsfeltet, så den sorte kant ikke puster accuracy op.

DRIVE's testsæt har intet facit. `test/` indeholder kun `images` og `mask`, og `test/mask` er FOV-masker. Det er kontrolleret: de dækker 68 % af billedet og overlapper træningssættets FOV-masker med IoU 0,97. Derfor laves alle splits ud fra de 20 annoterede træningsbilleder.

2 Splits

Lavet én gang med seed 42 og gemt i `splits/ph2.json` og `splits/drive.json`. Filerne ligger i git, så alle bruger præcis samme split. Del 2 genbruger PH2-splittet.

- **PH2:** 140 train, 30 val, 30 test.
- **DRIVE:** 14 train, 3 val, 3 test.

3 Kodestruktur

Alt køres fra `src/` via `main.py`.

Fil	Indhold
`config.py`	Stier, split-størrelser, standardværdier per datasæt, klik-indstillinger
`datasets/image_dataset.py`	Indlæsning, splits, `SegmentationDataset`, sparse klik-targets, datasæt-statistik
`datasets/transforms.py`	Resize og augmentation (geometri ens på billede/target/FOV, farve kun på billedet)
`datasets/clicks.py`	Simulering af klik: `random`, `interior`, `boundary`
`models/unet.py`, `encdec.py`	U-Net og EncDec med samme blokke; eneste forskel er skip connections
`training/losses.py`	BCE, Dice, focal, vægtet BCE, point-loss (balanceret og ubalanceret)
`training/trainer.py`	Træningsløkke, val efter hver epoch, gemmer bedste model på val-Dice
`utils/metrics.py`	Dice, IoU, accuracy, sensitivity, specificity per billede (evt. kun i FOV)
`utils/visualization.py`	Eksempelfigurer, klik-figur, Dice mod antal klik
`train.py`, `evaluate.py`	Én kørsel; evaluering på train/val/test og opsamling af alle kørsler
`main.py`	Kommandoerne `splits`, `clicks`, `stats`, `shapes`, `train`, `evaluate`, `summarize`
`jobs/part1.sh`, `part2.sh`	LSF-jobs til hele del 1 og hele klik-ablationen i del 2

U-Net

Standardopsætningen er 4 downsampling-niveauer og 32 basiskanaler, i alt 7,8 mio. parametre. Med et 256×256-input går den fra 32×256×256 ned til 512×16×16 i bottleneck og op igen til 1×256×256. Den fulde lag-for-lag-tabel til rapporten fås med kommandoen `shapes` (se nedenfor).

4 Sådan bruges det

```
git switch segmentation-pipeline     # indtil branchen er merget
uv sync                              # installerer afhængigheder i .venv
cd src
uv run python main.py train --dataset ph2 --model unet --loss focal
uv run python main.py train --dataset ph2 --loss point \
    --n-clicks 5 --click-strategy interior
uv run python main.py shapes --model unet --image-size 256   # U-Net-tabel
uv run python main.py summarize      # alle kørsler -> results/results.csv
```

Hele del 1 og del 2 submittes fra repo-roden. **Kør kun ét tungt job ad gangen i gruppen.**

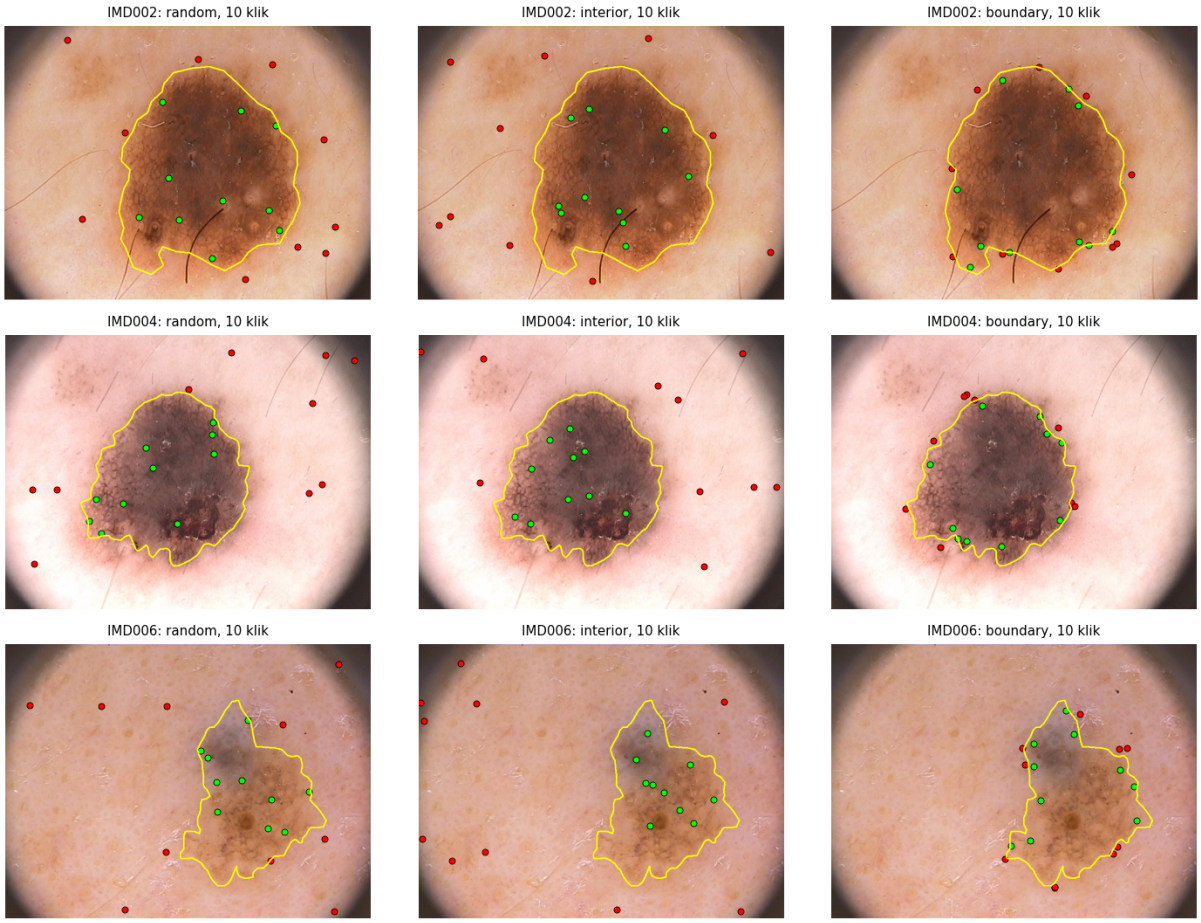
```
bsub < jobs/part1.sh   # EncDec + U-Net (BCE, focal, vægtet BCE) på PH2 og DRIVE
bsub < jobs/part2.sh   # U-Net med 1/3/5/10/20 klik x 3 strategier
```

Hver kørsel får en mappe i `runs/` (ikke i git) med `config.json`, `history.csv`, `best.pt`, `metrics.json`, metrikker per billede, forudsagte masker og `test_examples.png` (billede, ground truth, forudsigelse og fejlkort).

5 Valg der skal nævnes i rapporten

- **Opløsning:** PH2 resizes til 256×256. DRIVE resizes til 512×512, så de tynde kar bevares, og trænes på tilfældige 256-crops med hvert billede 10 gange per epoch. Metrikkerne beregnes i den resizede opløsning.

- **FOV:** I DRIVE beregnes metrikkerne kun inden for FOV-masken; loss beregnes på hele billedet.
- **Augmentation:** crops, flips, 90-graders rotationer og color jitter. Der er ingen interpolation, så masker og enkelte klik-pixels bevares præcist.
- **Træning:** modellerne giver logits, og der thresholdes ved `logits > 0` (= sigmoid $> 0{,}5$). Adam med lr 10^{-3}, 60 epochs. Bedste epoch vælges på val-Dice.
- **Focal loss:** standardversionen, hvor begge led vægtes med $(1 - p_t)^{\gamma}$, $\gamma = 2$.
- **Vægtet BCE:** `pos_weight` = #baggrund / #forgrund i træningssættet.
- **Klik:** 20 klik per klasse per billede er gemt per strategi i `clicks/`. `interior` ligger mindst 20 px fra læsionens kant, `boundary` højst 20 px. En kørsel med n klik bruger de første n, så klik-sættene er indlejrede. Point-loss er BCE kun på klikkede pixels, balanceret mellem positive og negative klik. Evaluering sker altid mod de fulde masker.
- **AI-brug:** pipelinen er skrevet med Claude Code og gennemgået af gruppen. Det skal med i rapportens afsnit om brug af AI-værktøjer.



Figur 1: De tre klik-strategier med 10 klik per klasse. Grøn: positive klik, rød: negative klik, gul: læsionens kant. Figuren ligger i `results/click_strategies.png`.

6 Hvad er kørt

- `splits`, `stats` (gemt i `results/dataset_stats.json`), `shapes` og `clicks`. Klik-figuren er gennemset, og klikkene ligger hvor de skal.

- Testkørsler på CPU med 1 epoch og lille billedstørrelse: PH2 med fuld supervision, PH2 med 3 klik og EncDec på DRIVE med vægtet BCE, plus `summarize`. Alt kørte uden fejl. Resultaterne er slettet, fordi de ikke kan bruges.
- **Ikke kørt:** rigtig træning på GPU. Jobscriptene er ikke submitted, og tidsforbruget per epoch på GPU kendes ikke endnu.

7 Næste skridt

1. Gennemgå branchen `segmentation-pipeline` og merge den ind i `main`.
2. Én person submitter `jobs/part1.sh` og holder øje med den første `.out`-fil (tid per epoch). Ved behov justeres `epochs` i `src/config.py`.
3. Når del 1 er færdig: submit `jobs/part2.sh`.
4. Kør `summarize` og brug `results/results.csv`, `dice_vs_clicks.png` og `test_examples.png` fra de enkelte kørsler i rapporten.